

App Developer Guide

From an empty Expo project to a live app on the Play Store, screen by screen.

46

Screens and sheets

24

Backend modules
consumed

12

Play Store release steps

9

Delivery phases

Stack

Expo • React Native •
TypeScript

Audience

Mobile (app) developers

Version

1.1 • 30 September 2026

Contents

1. Read this first	3
2. Tech stack	3
3. Project setup, step by step	4
4. Project structure and architecture	4
5. Design system and brand	5
6. API client rules	5
7. State, caching and realtime	7
8. Navigation, onboarding gate and deep links	7
9. Screen catalogue	9
10. Feature implementation notes	11
11. Error code → UI mapping	12
12. Permissions, push channels and analytics	13
13. Security and privacy in the app	13
14. Quality: performance, offline, accessibility	14
15. Testing	14
16. Builds, environments and versioning	15
17. Releasing to the Play Store	15
18. After release: monitoring, OTA and hotfixes	15
19. iOS (later)	15
20. Delivery plan, workflow and Definition of Done	17

1 Read this first

This guide tells an app developer how to build the Kuchu Puchu mobile app, from an empty project to a production release on Google Play. The app is **Android first**; the same codebase ships to iOS later (section 19). Every screen names the backend module (M01–M24) and the APIs it uses. Exact request and response shapes are in the Backend Developer Guide and in the OpenAPI file at `/api/docs` on staging.

46

screens & sheets

~60

API endpoints used

8

realtime events

5

push channels

3

build profiles

Golden rules

Rule	Why
The server is the source of truth	The app never decides entitlements, verification results, match state or who can see whom. It only displays what the API returns.
Store almost nothing on the phone	Only the refresh token and device id are persisted (secure storage). No phone numbers, messages or photos in logs or analytics.
Every screen has 4 states	Loading, content, empty and error with retry. QA tests all four.
Handle errors by code, not by message	Messages can change; codes are stable (section 11).
Design for low-end Android	2–3 GB RAM phones and slow networks are the majority. Test on real devices, not only emulators.
Never reveal a block	Blocked, hidden and deleted people all look like "This profile is no longer available".

2 Tech stack

Area	Choice	Notes
Framework	Expo (current SDK), React Native, TypeScript strict	Use a development build (not Expo Go) because of native SDKs
Navigation	Expo Router (file-based)	Route groups: (auth), (onboarding), (app)
Server state	TanStack Query	Caching, retries, cursor pagination with <code>useInfiniteQuery</code>
Local state	Zustand	Session, discovery queue, UI flags
Forms	react-hook-form + zod	Validation mirrors the backend; the server stays authoritative
Secure storage	expo-secure-store	Refresh token and device id only
HTTP	fetch wrapper (or ky)	One client with interceptors (section 6)
Realtime	socket.io-client	WebSocket transport only
Camera & images	expo-camera, expo-image-picker, expo-image-manipulator, expo-image	HEIC → JPEG before upload; cache images by photo id
Liveness	Provider SDK (AWS Face Liveness or HyperVerge/IDfy) through a config plugin	Backend chooses the provider (M11)
OTP autofill	Android SMS Retriever (e.g. react-native-otp-verify)	SMS must contain the app hash; add it to the DLT template
Location	expo-location	Foreground only, never background
Push	expo-notifications with the native FCM device token	Token sent to M20
Payments	react-native-iap (Google Play Billing)	Every purchase verified by the backend (M21)
Calls	Agora or 100ms React Native SDK	Phase 8 (M24)
Lists	@shopify/flash-list	Chat, matches, notifications
Animation & gestures	react-native-reanimated, react-native-gesture-handler	Card swipes at 60 fps
Screen protection	expo-screen-capture	Block screenshots on sensitive screens
Monitoring	Sentry (@sentry/react-native)	PII scrubbing on
Build & release	EAS Build, EAS Submit, EAS Update	OTA only for JS fixes
Tests	Jest, React Native Testing Library, Maestro	

3 Project setup, step by step

```
# 1. create the project (TypeScript + Expo Router template)
npx create-expo-app@latest mobile-app
cd mobile-app

# 2. core libraries
npx expo install expo-router expo-secure-store expo-image expo-image-picker expo-image-manipulator \
  expo-camera expo-location expo-notifications expo-screen-capture expo-device expo-crypto \
  react-native-reanimated react-native-gesture-handler
npm i @tanstack/react-query zustand react-hook-form zod @hookform/resolvers socket.io-client \
  @shopify/flash-list react-native-iap @sentry/react-native
npm i -D openapi-typescript jest-expo @testing-library/react-native eslint prettier

# 3. EAS
npm i -g eas-cli && eas login && eas init
eas build --profile development --platform android      # dev client with native modules
npx expo start --dev-client

# 4. generate API types from staging
npx openapi-typescript https://api.staging.kuchupuchu.in/api/docs-json -o src/api/schema.ts
```

Setup item	Details
Package name	in.kuchupuchu.app (example). Never changes after the first Play upload.
app.config.ts	Name, slug, scheme kuchupuchu, Android package, permissions (section 12), plugins (camera, location, notifications, liveness SDK, IAP, Sentry), extra values per environment.
Environment values	API_URL, SOCKET_URL, SENTRY_DSN, APP_ENV from EAS environment variables. No secrets in the app.
Lint and format	ESLint (expo config) + Prettier, run in CI and a pre-commit hook.
CI	GitHub Actions: type-check, lint, tests, and API type generation check on every pull request.
Local backend	Point API_URL to the backend dev server or docker-compose from the Backend Guide.

4 Project structure and architecture

```
mobile-app/
├── app/                                Expo Router routes
│   ├── (auth)/                        welcome, sign-in, verify-otp
│   ├── (onboarding)/                 consent, selfie, photos, basics, about, interests, prompts, preferences, permissions
│   ├── (app)/(tabs)/                 discover, likes, matches, profile
│   ├── (app)/chat/[matchId]          conversation, call
│   ├── (app)/settings/               account, privacy, notifications, blocked, devices, subscription, delete
│   └── _layout.tsx                   providers + gate (section 8)
├── src/
│   ├── api/                          http client, endpoints per module, error mapping, schema.ts (generated)
│   ├── auth/                         SessionManager, token storage, device id
│   ├── realtime/                     socket client, event → cache handlers
│   ├── features/<area>/               screens logic, hooks and components per feature
│   ├── components/                   design system (section 5)
│   ├── theme/                        colour tokens, typography, spacing
│   ├── i18n/                         en.json (Hindi-ready)
│   └── lib/ · hooks/ · config/
└── __tests__/ · .maestro/            unit and e2e tests
```

- **Layers:** screen (UI only) → feature hook (useSendMessage, useDiscoveryQueue) → API endpoint function → HTTP client. Screens never call fetch directly.
- **Feature folders** own their screens, hooks and components. Shared UI lives only in components/.
- **Generated types** from the backend OpenAPI file make a backend contract change fail the app build, not production.
- **One SessionManager** owns tokens, refresh, logout, socket lifecycle and push registration.

5 Design system and brand

5.1 Brand colours

Token	Hex	Use
 <code>color.primary</code>	#c76883	Primary buttons, like button, active tab, links, badges, progress, highlights
 <code>color.ink</code>	#111111	Text, icons, headers, pass button outline, dark-mode background
 <code>color.surface</code>	#ffffff	Main background (light mode), text on pink and black
 <code>color.primarySoft</code>	#fff8f8	Cards, input backgrounds, chips, selected states, info banners

- Derive only transparent variants of these four colours (for example ink at 60% for secondary text, primary at 20% for borders). Do not add new hues, except a system red for destructive actions and errors, required for clarity.
- Text on #c76883 must be white and at least 16 px bold (buttons) to keep good contrast. Body text is always #111111 on white or #fff8f8.
- Dark mode: background #111111, text white, primary stays #c76883, cards white at 6% opacity.

5.2 Typography, spacing and shape

Token	Value
Font	One family (for example Inter or Poppins, bundled with the app). Weights 400, 600, 700.
Type scale	Display 28 • Title 22 • Heading 18 • Body 16 • Label 14 • Caption 12 (sp). Respect system font scaling.
Spacing	4-point grid: 4, 8, 12, 16, 24, 32. Screen side padding 16.
Radius	Buttons and inputs 12 • cards 16 • profile card 20 • chips full round
Elevation	Soft shadow only on the discovery card and bottom sheets
Icons	One outline icon set, 24 px, colour ink or primary
Motion	150–250 ms ease-out; card swipe with spring physics; respect "reduce motion"

5.3 Base components (build before feature screens)

Component	Variants and rules
Button	primary (pink), secondary (black outline), ghost, destructive; loading and disabled states; min height 48
TextField / OtpInput	label, helper, error text; OTP 6 boxes with paste and autofill
Chip / Tag	selectable (interests), status (approved, in review, rejected)
Card / ProfileCard	photo carousel with dots, name + age, city + distance bucket, verified badges
Avatar	sizes 32 / 48 / 72, blurred variant for "Likes you"
Badge	face-verified tick, ID-verified shield, plan badge, unread count
Sheet / Modal / Dialog	bottom sheet for menus and contact exchange; dialog for confirmations
Banner / Toast	info (soft pink), error, offline
EmptyState / ErrorState / Skeleton	every list and screen uses these
PaywallCard	plan name, price, entitlement list from the API, primary CTA

6 API client rules

Topic	Rule
Base URL	From environment config (section 16). Always HTTPS outside local development.
Headers	Authorization: Bearer, X-App-Version, X-Platform: android, X-Device-Id (stable per install: android:<uuid>)
Envelope	Success {success, data, meta} → return data. Error {success:false, code, message, details, requestId} → throw ApiError.
Tokens	Access token in memory only. Refresh token in secure storage. On app start: refresh first, then load /auth/me.
401 handling	Single-flight refresh: the first 401 triggers one refresh; other requests wait and retry once. Refresh fails → clear session → sign-in screen.
Special codes	ONBOARDING_INCOMPLETE → route to details.nextStep • ENTITLEMENT_REQUIRED → paywall • ACCOUNT_RESTRICTED → restricted screen • REAUTH_REQUIRED → OTP sheet
Idempotency	Each chat message gets a clientMessageId (UUID) kept across retries. Purchases send an Idempotency-Key.
Retries	Only GET and idempotent writes retry automatically (max 2, exponential backoff). Never auto-retry a purchase without its key.

Topic	Rule
Pagination	Pass <code>meta.nextCursor</code> back as <code>cursor</code> . Chat uses <code>beforeSeq</code> / <code>afterSeq</code> .
Force update	Call <code>/app/config</code> on launch; below <code>minVersion</code> → blocking update screen with a Play Store link.
Timeouts	15 s default, 60 s for photo upload.
Support	Show <code>requestId</code> on the generic error screen ("Error code: ...") so support can find it in logs.

7 State, caching and realtime

7.1 Query keys and invalidation

Query key	Source	Invalidate / update when
['me']	GET /auth/me	Login, selfie approved, profile created, plan changed
['profile'], ['completion']	M09	Any profile, photo, interest or prompt change
['photos']	M12	Upload, reorder, delete, moderation push
['preferences']	M10	Save → also flush the discovery queue
['discovery']	M18	Kept in Zustand queue, not in the query cache
['likes', 'received']	M17	Push like.received, plan change
['matches']	M16	Socket match:new, match:ended, message:new (reorder)
['messages', matchId]	M19	Socket message:new, message:deleted; re-sync with afterSeq
['billing', 'me']	M21	After purchase verify, restore, app resume
['notifications']	M20	Push received, mark read

7.2 Realtime events

Event (server → app)	App action
message:new	Insert by seq; if seq has a gap, fetch afterSeq; update match list preview and unread badge
message:deleted	Replace with tombstone "Message unsent"
match:new	Add to matches; show "It's a match" if the user is in the app
match:ended	Remove from list; if the chat is open, show "This conversation is no longer available" and go back
match:read	Update read ticks (only if the partner allows receipts)
typing	Show typing dots for 4 s
contact:updated	Refresh the contact-exchange state in the chat
call:incoming	Show the incoming call screen (Phase 8)

- Connect after login with auth: {token}; on auth_expired disconnect → refresh token → reconnect.
- App to background: keep the socket up to 30 s, then disconnect (push takes over). On foreground: reconnect and re-sync open chats and the match list.
- Client sends only typing (max once every 3 s). Every write goes through REST.

8 Navigation, onboarding gate and deep links

The root layout decides where the user goes each time the app opens or /auth/me changes:

```
no session ..... (auth)/welcome
session + nextStep = selfie ..... (onboarding)/consent → selfie
nextStep = photos ..... (onboarding)/photos
nextStep = profile ..... (onboarding)/basics → about → interests → prompts
nextStep = preferences ..... (onboarding)/preferences → location → notifications
nextStep = done ..... (app)/(tabs)/discover
status suspended / banned ..... restricted screen (appeal if allowed)
app below min version ..... force-update screen
```

- Progress comes from the server, so the user can close the app at any step and resume at the same step on any device.
- Tabs: **Discover**, **Likes**, **Matches** (chat inside), **Profile**. Settings open from Profile.

Deep link	Opens	Used by
kuchupuchu://match/<matchId>	It's-a-match / chat	push match.new
kuchupuchu://chat/<matchId>	Conversation	push message.new
kuchupuchu://likes	Likes you	push like.received
kuchupuchu://verification	Verification centre	selfie and ID results
kuchupuchu://settings/devices	Devices	new-device security alert
https://kuchupuchu.in/app/...	Same targets (Android App Links)	email links

Always re-check access through the API after opening a deep link; a match may have ended since the push was sent.

9 Screen catalogue

46 screens and sheets. API paths are relative to `/api/v1`. Each screen also has loading, empty and error states (not repeated below).

9.1 Auth M06 • 3 screens

#	Screen	APIs (module)	Behaviour, validation and states
1	Welcome	GET <code>/app/config</code>	Brand, "18+ only", Continue button, terms and privacy links; checks force update
2	Sign in	POST <code>/auth/otp/request</code>	Phone tab (+91 prefix, 10 digits) or email tab; validate format before sending; always shows "Code sent" (the server never reveals if an account exists); 429 → countdown from <code>retryAfterSeconds</code>
3	Verify OTP	POST <code>/auth/otp/verify</code> , <code>/auth/otp/request</code>	6 boxes with SMS Retriever autofill and paste; resend enabled after 60 s; wrong code → shake + message; after 5 wrong tries ask for a new code; success stores tokens and routes by <code>nextStep</code>

9.2 Onboarding M11, M12, M09, M10 • 11 screens

#	Screen	APIs (module)	Behaviour, validation and states
4	Consent	—	Explains live selfie: why, how long kept, never shown to others; checkbox + Continue; sends <code>consentVersion</code> with the session
5	Live selfie	POST <code>/verification/face/session</code> , <code>/face/complete</code>	Tips (good light, no mask, glasses off); provider SDK camera; result: approved → photos • in review → screen 6 • rejected → reason + attempts left + try again
6	Selfie in review	GET <code>/verification</code>	"Usually under 2 hours"; polls every 60 s and listens for push; user may continue to photos while waiting
7	Photos	GET/POST <code>/profile/photos</code> , PUT <code>/order</code> , DELETE	6-slot grid, minimum 4 approved; camera or gallery; HEIC→JPEG, resize ≤ 2048 px; progress bar and cancel; status badge per photo; long-press drag to reorder; primary must match verified face (422 PHOTO_FACE_MISMATCH)
8	Basics	POST <code>/profile</code>	Name (2–50), date of birth (age preview, 18+; "cannot be changed later" confirm), gender + show-gender switch
9	About you	PATCH <code>/profile</code>	Bio 500, occupation, education, intent, height, languages; skip allowed; 422 CONTACT_DETAILS_NOT_ALLOWED highlights the field
10	Interests	GET <code>/catalog/interests</code> , PUT <code>/profile/interests</code>	Chips by category, pick 3–10, counter shown
11	Prompts	GET <code>/catalog/prompts</code> , PUT <code>/profile/prompts</code>	Choose up to 3 questions, answer 200 chars; skip allowed
12	Preferences	PUT <code>/preferences</code>	Age range slider (18–100), genders multi-select, distance 1–200 km; paid filters shown with a lock → paywall
13	Location	PUT <code>/profile/location</code>	Explainer: "only your approximate area is used"; foreground permission; denied → city picker fallback
14	Notifications	PUT <code>/notifications/devices</code>	Explainer, then OS prompt (Android 13+); skip allowed; finish → Discover

9.3 Discover, likes and matches M18, M17, M16, M21 • 10 screens

#	Screen	APIs (module)	Behaviour, validation and states
15	Discover	GET <code>/discovery</code> , POST <code>/likes</code> , POST <code>/passes</code>	Card stack: photo carousel, name + age, city, distance bucket, badges, top prompt; swipe or buttons (like pink, pass black outline); queue refills below 3 cards with <code>exclude</code> ; 409 DISCOVERY_NOT_READY → fix-up list from <code>details.missing</code>
16	Profile detail	GET <code>/profiles/:userId</code>	All photos, prompts, shared interests highlighted; menu: report, block; 404 → "no longer available"
17	Filters	PUT <code>/preferences</code>	Bottom sheet; saving flushes the local queue and refetches
18	It's a match	POST <code>/likes</code> response, socket <code>match:new</code>	Both photos, "Send a message" / "Keep browsing"
19	Out of likes	GET <code>/billing/me</code>	Reset time from <code>retryAfterSeconds</code> , plan comparison, restore the card that failed
20	Undo pass	DELETE <code>/passes/:targetUserId</code>	Button visible only with <code>undo_pass</code> ; 5-minute window
21	Likes you	GET <code>/likes/received</code>	Free/Basic: count + blurred grid + upgrade; Plus/Premium: cards with like/pass

#	Screen	APIs (module)	Behaviour, validation and states
22	Matches	GET /matches	Row of new matches, then conversations by latest activity; unread badges; previews ("Photo" or text ≤ 100 chars); realtime updates
23	Empty discovery	—	"No one new nearby" with "Widen distance" and "Widen age range" actions
24	Boost	POST /boosts/activate	Remaining boosts, 30-minute countdown while active

9.4 Chat, contacts and calls M19, M22, M24 • 7 screens

#	Screen	APIs (module)	Behaviour, validation and states
25	Conversation	GET/POST /matches/:id/messages, /read, socket	Optimistic bubbles (sending → sent / failed + retry) with <code>clientId</code> ; history with <code>beforeSeq</code> ; re-sync with <code>afterSeq</code> ; typing dots; links as plain text with "Open link" warning sheet; long-press own message → unsend (15 min); first-message tips for new matches; 429 MESSAGE_AWAITING_REPLY → "Wait for a reply"
26	Chat menu	DELETE /matches/:id, /blocks, /reports	View profile, mute, unmatched (confirm), report, block
27	Contact exchange	POST/GET /matches/:id/contact-exchange	Checklist of conditions (24 h, 3 messages each, both verified); paywall without entitlement; pending state
28	Contact request	POST /contact-exchanges/:id/accept decline	Received request card; warning "a shared number cannot be taken back"; accept is free
29	Contact card	message type <code>contact_card</code>	Number with Call and WhatsApp buttons; hidden after unmatched or block; screenshots blocked
30	Contact blocked hint	422 CONTACT_SHARING_LOCKED	Explains why the message was not sent and offers contact exchange; keeps the text in the input
31	Call	POST /matches/:id/calls, PATCH /calls/:id	Outgoing, incoming (full-screen), in-call controls (mute, speaker, camera, end); Phase 8

9.5 Profile, settings and safety M09, M07, M06, M13–M15, M20, M21, M23 • 15 screens

#	Screen	APIs (module)	Behaviour, validation and states
32	My profile	GET /profile, /profile/completion	Preview as others see it; completion ring; tips from <code>missingFields</code>
33	Edit profile	PATCH /profile and related	Same forms as onboarding, saved per section; DOB read-only
34	Verification centre	GET /verification, POST /verification/id/start	Face status; optional ID badge: choose document → provider flow → result; screenshots blocked
35	Plans (paywall)	GET /billing/plans, POST /billing/purchases/verify	Plan cards (Free, Basic ₹199, Plus ₹499, Premium ₹999) with entitlements from the API; monthly/quarterly toggle; purchase flow 10.5; auto-renew and cancel text
36	My subscription	GET /billing/me, POST /billing/restore	Plan, renewal date, remaining quotas; "Manage in Google Play"; restore purchases
37	Settings	—	Sections, app version, sign out
38	Privacy	PATCH /profile	Snooze, hide profile, incognito (Premium), read receipts, allow calls, show gender
39	Notification settings	GET/PATCH /notifications/preferences	Matches, messages, likes, message previews; security shown as always on
40	Notifications inbox	GET /notifications, /read	List with deep links; mark all read; unread badge
41	Blocked users	GET /blocks, DELETE /blocks/:id	Unblock with confirmation ("match will not come back")
42	Devices	GET /auth/sessions, DELETE, /logout-all	Current device marked; sign out one or all
43	Report	POST /reports	Category → optional details (1000) → "Also block" on by default → thank-you screen; never shows the outcome
44	Safety centre	static	Dating safety tips, helplines, how reporting and verification work
45	Delete account	POST /auth/reauth/*, DELETE /account	What gets deleted → OTP step-up → reminder to cancel the Google Play subscription → signed out; also data export option
46	Restricted / appeal	POST /account/appeals	Suspension or ban reason category and end date; appeal form when allowed

10 Feature implementation notes

10.1 Session manager

- States: unknown → signedOut → signedIn. On launch: read refresh token → /auth/refresh → /auth/me. Offline at launch → show cached shell with an offline banner, retry when online.
- Logout, refresh failure or account deletion: clear secure storage, React Query cache, Zustand stores, image cache, socket, and delete the push device on the server.

10.2 OTP sign-in

- Normalise phone before sending (strip spaces, add +91). The server is still authoritative.
- Keep the resend countdown across app restarts by storing only the "next allowed at" time in memory for the session.

10.3 Live selfie

- Flow: consent → create session (server) → provider SDK → /face/complete → show the **server** decision. Never show "verified" based on the SDK result alone.
- Handle: camera permission denied (settings deep link), app backgrounded mid-capture (restart session), poor light, VERIFICATION_ATTEMPTS_EXCEEDED (support link).

10.4 Photos

- Before upload: convert HEIC to JPEG, resize the longest edge to 2048 px, quality 0.85. Upload multipart with progress and cancel.
- Only approved photos count toward the minimum of 4. Show the server status and rejection reason on each photo.
- Cache with expo-image using cacheKey = photoId + size; signed URLs change every hour. Use thumb in lists, medium on cards, large only full screen.

10.5 Purchases (Google Play Billing)

```
1 GET /billing/plans → product ids → react-native-iap: fetch subscriptions (with offer tokens)
2 user taps a plan → request subscription (productId, offerToken)
3 purchase update → POST /billing/purchases/verify {store, productId, purchaseToken} + Idempotency-Key
4 server verifies with Google and ACKNOWLEDGES → app finishes the transaction only after server success
5 refresh ['billing', 'me'] and ['me'] → unlock UI
6 on every app start: check pending/unfinished purchases and repeat step 3 (a network failure must never lose a purchase)
```

- Never unlock features from the client purchase event alone. Show "Activating your plan..." while step 3 runs.
- Upgrades and downgrades use Play's replacement modes; the backend decides the resulting plan from the webhook.

10.6 Discovery queue

- Zustand queue of cards. Prefetch images of the next 3 cards. Refill when below 3, sending the remaining ids as exclude.
- Like/pass removes the card optimistically; 404 → drop silently; 429 → restore the card and show Out of likes; match in response → It's a match.

10.7 Chat

- Messages stored by seq per match. Pending messages have no seq yet and are keyed by clientId.
- Send failure: keep the bubble with "Tap to retry" (same clientId, so no duplicates).
- Mark read when the newest message is visible and the app is in foreground: POST /read {lastReadSeq}, debounced 1 s.
- Socket down while a chat is open: poll afterSeq every 5 s until it reconnects.

10.8 Push notifications

- After permission, get the native device token and call PUT /notifications/devices on every launch and on token change.
- Foreground push for the open chat is not shown (the socket already delivered it). Message pushes show no text unless the user enabled previews.

11 Error code → UI mapping

Code (HTTP)	What the user sees / what the app does
VALIDATION_ERROR (400)	Field errors under inputs; generic "Please check the highlighted fields"
UNAUTHORIZED (401)	Silent refresh once; if it fails → sign-in screen
OTP_INVALID (401)	"That code isn't right or has expired"
ACCOUNT_RESTRICTED (403)	Restricted screen (46)
REAUTH_REQUIRED (403)	OTP step-up sheet, then repeat the action
ONBOARDING_INCOMPLETE (403)	Route to <code>details.nextStep</code>
ENTITLEMENT_REQUIRED (403)	Paywall highlighting <code>details.entitlement</code>
USER / PROFILE / MATCH_NOT_FOUND (404)	"This profile/conversation is no longer available"; remove it from lists
PHOTO_LIMIT_REACHED • DISCOVERY_NOT_READY • INTERACTION_ALREADY_LIKED (409)	Explain and offer the fix (delete a photo / finish setup / nothing)
CONTACT_DETAILS_NOT_ALLOWED (422)	"For your safety, contact details can't be added here"
CONTACT_SHARING_LOCKED (422)	Screen 30 with contact-exchange offer
PHOTO_FACE_MISMATCH (422)	"Your main photo must clearly show you"
PHOTO_INVALID_FILE (400)	"Use a JPG or PNG photo, at least 400×400, under 10 MB"
OTP_COOLDOWN • TOO_MANY_REQUESTS (429)	Countdown from <code>retryAfterSeconds</code>
LIKE_LIMIT_REACHED (429)	Out of likes screen (19)
MESSAGE_AWAITING_REPLY (429)	"Give them a chance to reply"
VERIFICATION_ATTEMPTS_EXCEEDED (429)	"Try again tomorrow" + support link
PURCHASE_INVALID / ALREADY_LINKED (422)	"We couldn't activate this purchase" + restore + support
5xx / network	Generic error with <code>retry</code> and <code>requestId</code> ; offline banner for network errors

12 Permissions, push channels and analytics

12.1 Permissions

Permission	When asked	If denied
Camera	Selfie screen, add photo from camera	Selfie cannot continue → settings link; photos from gallery still work
Photos (Android photo picker)	Add photo from gallery	Picker needs no permission on new Android; older: ask <code>READ_MEDIA_IMAGES</code>
Location (foreground, approximate or precise)	Onboarding screen 13	City picker fallback
Notifications (Android 13+)	Onboarding screen 14	App works; banner in Matches suggests enabling
Microphone	First call (Phase 8)	Voice calls unavailable

Never request: background location, contacts, SMS read, phone state, storage write. Play reviewers reject unexplained permissions.

12.2 Android notification channels

Channel	Importance	Notifications
Matches	High	New match
Messages	High	New message (grouped per match)
Likes	Default	Someone likes you (max 1 per 6 h)
Account & security	High	New device, suspicious sign-in, verification results, moderation notices
Calls	Max (full-screen)	Incoming call (Phase 8)

12.3 Analytics events (no personal data)

Event	Properties
<code>onboarding_step_completed</code>	<code>step</code> , <code>duration_s</code>
<code>selfie_result</code>	<code>result</code> (approved / review / rejected), <code>attempt_no</code>
<code>discovery_card_action</code>	<code>action</code> (like / pass / undo), <code>position_in_queue</code>
<code>match_opened</code> • <code>message_sent</code>	<code>is_first_message</code>
<code>paywall_viewed</code> • <code>purchase_started</code> • <code>purchase_completed</code>	<code>source screen</code> , <code>plan</code> , <code>period</code>
<code>report_submitted</code> • <code>block_used</code>	<code>category</code> (report only)
<code>app_error</code>	<code>error code</code> , <code>screen</code>

Never send user ids of other people, names, messages, locations or phone numbers to analytics. Use the backend user id only if the privacy policy allows it.

13 Security and privacy in the app

#	Rule
1	Refresh token only in <code>expo-secure-store</code> (<code>WHEN_UNLOCKED_THIS_DEVICE_ONLY</code>). Access token only in memory.
2	No PII, tokens, OTPs, message text or photo URLs in logs, Sentry breadcrumbs or analytics.
3	Block screenshots (<code>expo-screen-capture</code>) on selfie, contact card, ID and delete-account screens.
4	Location: foreground only, sent at most every 15 min; never display or store anyone's coordinates.
5	Release builds: no dev menu, no debug endpoints, HTTPS only, <code>allowBackup=false</code> , cleartext traffic disabled, code minified (Hermes + R8).
6	Links in chat are never auto-opened; the warning sheet shows the full domain first.
7	Clear everything on logout and deletion, including cached images of other users.
8	Explain every permission on a screen before the OS prompt; ask only when the feature is used.
9	Do not embed any secret key in the app. Only public identifiers (Sentry DSN, provider public keys).
10	Optional hardening later: certificate pinning, root/emulator detection signals sent to the backend (never block silently).

14 Quality: performance, offline, accessibility

Area	Target / rule
Cold start	< 2.5 s on a mid-range phone (D2 in the QA Guide)
Card swipe	60 fps, gestures on the UI thread (Reanimated)
Lists	FlashList, fixed item heights where possible, images at thumb size
App size	AAB download < 40 MB; remove unused fonts and assets
Memory	No crash in a 30-minute session on a 2 GB RAM phone
Offline	Offline banner; cached matches and messages visible; unsent messages retry when online
Accessibility	Labels on all buttons, font scaling to 130% without broken layouts, contrast AA, like/pass as buttons (not swipe only), TalkBack order checked
Localisation	All strings in <code>i18n/en.json</code> ; locale-aware dates; layouts tolerate longer Hindi text

15 Testing

Level	What to test
Unit (Jest)	API client and error mapping, SessionManager refresh race, validation schemas, seq gap logic, discovery queue, entitlement UI helpers
Component (RNTL)	Every screen's four states; forms with invalid input; locked paid features; error mapping
E2E (Maestro)	Sign in → selfie (mock provider on staging) → photos → profile → discover → like → match → chat → report → delete account; purchase with a license tester
Manual devices	QA device matrix (low RAM, Android 10, small screens, slow 3G)

Staging provides test phone numbers with a fixed OTP and a mock liveness provider so automation can run. These never exist in production.

16 Builds, environments and versioning

EAS profile	API	Output	Use
development	local / dev	Dev client APK	Daily development with debugging
preview	staging	APK	QA builds, internal sharing
production	production	AAB	Play Store (internal → closed → production tracks)

Topic	Rule
Config	<code>app.config.ts</code> reads <code>APP_ENV</code> and sets API URL, socket URL, Sentry DSN, app name suffix ("Kuchu Puchu Staging") and a separate package suffix for non-production builds so both can be installed together.
Signing	EAS manages the upload key; Play App Signing enabled. Back up credentials with <code>eas credentials</code> .
Versioning	<code>version semantic</code> (1.4.0) shown to users; <code>versionCode</code> auto-increments on every production build.
OTA updates	EAS Update channels per profile, runtime version tied to the native build. OTA only for JS-only fixes; any native change needs a store build.
Changelog	Every release has user-facing notes (Play) and internal notes (tickets, backend version needed).

17 Releasing to the Play Store

Step	What to do
1. Developer account	Create an organisation Google Play Console account (needs the company's D-U-N-S number). Personal accounts created after late 2023 must run a closed test with at least 12 testers for 14 days before production.
2. Create the app	App name, default language English (India), App, Free (with in-app purchases). Package name from section 16.
3. App signing	Enable Play App Signing; upload the first AAB (<code>eas build --profile production</code> , <code>eas submit</code>) to the internal testing track.
4. Store listing	Short description (80 chars), full description, icon 512 px, feature graphic 1024×500, 4–8 phone screenshots, privacy policy URL, support email, website. Use the brand colours for all graphics.
5. App content	Privacy policy • Ads: none • App access : reviewer test phone number with fixed OTP and a pre-verified profile (production reviewer account only) • Content rating questionnaire (social/dating, user interaction) • Target audience: 18+ only • News/government/financial: no.
6. Data safety form	Collected: phone/email, name, date of birth, gender, photos, approximate location, messages, purchase history, device ids. Purpose: app functionality, safety, fraud prevention. Encrypted in transit. Users can request deletion. Not sold.
7. Account deletion	Play requires in-app deletion and a public web page to request deletion. Add the web URL from the Deployment Guide.
8. Permissions	Declare camera, photos, foreground location, notifications, microphone (calls). No background location, no SMS permissions (SMS Retriever needs none).
9. Dating & UGC policy	Report and block visible in profile and chat, moderation in place, community guidelines published, 18+ enforced.
10. Monetization	Payments profile and merchant account; subscription products with ids from the backend <code>plan_products</code> (monthly, quarterly, half-yearly base plans); license testers added; RTDN Pub/Sub topic linked (backend M21).
11. Testing tracks	Internal (team) → Closed (beta users in the launch city) → Production. Fix every crash in the pre-launch report.
12. Production rollout	Staged rollout 10% → 50% → 100% while watching crash-free rate ($\geq 99.5\%$) and ANR rate. Halt the rollout if either drops.

18 After release: monitoring, OTA and hotfixes

- Watch Sentry and Play Console vitals daily for the first week after each release (crashes, ANRs, slow start).
- Hotfix path: JS-only bug → EAS Update to the production channel (after QA smoke on preview) • native bug → new build, expedited review, staged rollout.
- Keep the backend backward compatible with the two latest app versions; raise `minVersion` in `/app/config` only when an old version is unsafe.
- Reply to Play reviews about bugs within 48 h; copy technical ones into the bug tracker.

19 iOS (later)

- Same codebase. Needs an Apple Developer Program organisation account, App Store Connect app and StoreKit products mirroring Play products.
- Push through APNs (key in Firebase), SMS autofill works through iOS one-time-code suggestions, photo picker and camera permission strings in `Info.plist`.

- Review points: in-app account deletion, UGC moderation (report, block, filtering), privacy nutrition labels, subscription terms on the paywall.

20 Delivery plan, workflow and Definition of Done

Phase	App work	Backend needed
0	Project setup, design system with brand colours, API client, SessionManager, CI	M01–M03
1	Welcome, sign in, OTP, devices, delete account, force update	M04–M07
2	Consent, selfie, photos, basics, about, interests, prompts, preferences, permissions	M08–M12
3	Report and block flows, safety centre, restricted screen	M13–M15
4	Discover, profile detail, filters, likes, matches list, match modal	M16–M18
5	Chat, realtime, push, inbox, notification settings	M19–M20
6	Paywall, subscription, contact exchange, ID badge, boost, incognito, travel mode	M21–M23
7	Hardening: accessibility, performance, offline, beta, Play release	—
8	Calls	M24

Workflow

- Branches feature/screen-25-conversation; pull request with screenshots or a short video, linked ticket and test notes.
- PR checklist: 4 states done • strings in i18n • no PII in logs • works in dark mode and at 130% font • tested on a low-end device • tests added.
- A screen is started only when its backend endpoints exist on staging (or are mocked from the OpenAPI file).

Done when (every screen): matches Figma and the brand palette • 4 states implemented • errors mapped by code • analytics events without PII • accessibility checked • unit/component tests written • QA test cases passed on the device matrix.